

Parallel Scaling

Ana Costa Pereira, Joshua Davies

26/06/2026



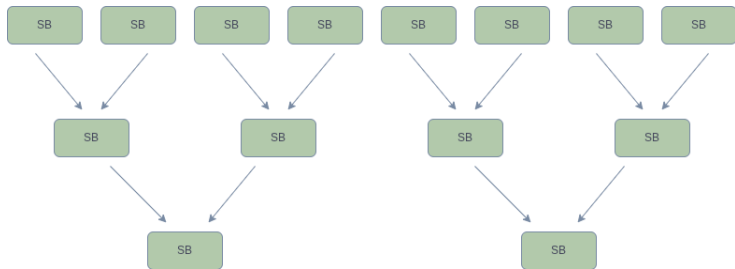
1 Parallel scalings

- Parallel scalings: study of scaling at higher thread number of tform;

Outline of talk

- Overview of sorting process in parallel;
- Benchmarks performance at different thread count;
- Changes implemented in sorting of terms;

Parallel Sorting in TForm



- Master makes the allocations for the workers;
- For each N workers, $N - 2$ SB are created;

- Each worker performs a local sort of its generated terms.
- Workers produce independent sorted streams.
- Sortbots merge pairs of sorted outputs in parallel.
- The master performs only the final merge and writes output.
- Goal: reduce master-thread bottlenecks and maximize worker utilization.

- Memory is shared globally between all threads;
- Avoid overwriting: each thread makes a private copy of the data: allocation of $1/N$ of the master's buffer size
- Fast in-memory sorting starts in the buffers.
- When buffers fill up, data is spilled to disk.

Working of the code:

- Terms are allowed to overlap between blocks;
- Reading thread always locks pair of locks - the current thread and the previous thread;
- 0th block - the overlap in the end of term is copied to 0th so that the terms are contiguous - this is no longer needed when we impose that only full terms can fit into the blocks

Data Fragmentation Strategy:

- **Overlapping Blocks:** Raw data streams permit terms to cross block boundaries to maximize ingestion speed.
- **Optimization (Structural Change):** Realizing terms can overlap requires complex boundary tracking. The modern code enforces that *only complete terms* sit inside blocks—rendering historical 0th-block stitching obsolete.

The "Chained" Locking Mechanism:

- Threads process blocks sequentially but execute in parallel.
- To safely read an overlap boundary, the reading thread must employ a **Hand-over-Hand (Sliding Window)** lock.

Visualizing Lock & Release (Hand-over-Hand)

- Step 1:** Thread N starts $\rightarrow$ [Lock Thread $N - 1$] [Lock Thread N]
- Step 2:** Thread N safely reads data block and overlapping boundary.
- Step 3:** Thread N finishes $\rightarrow$ [Release Thread $N - 1$] [Release Thread N]

Why this design works:

- **No Race Conditions:** Thread $N - 1$ cannot destroy or alter its trailing boundary data while Thread N is reading it.
- **Prevents Deadlocks:** Because threads always acquire locks in strict chronological order ($N - 1$ then N), a deadlock loop is mathematically impossible.

Before changes

- Default buffer sizes were increased over time
- Size of sortbots increased;
- The output from a worker fits in a single block → no advantage on running in parallel;
- Each stage of sortbot merge tree runs one after the other
- Terms can be partially shared between blocks: when a term is being written in a block and the block finishes, the term keeps being written into the next block

After changes

- Blocks are made smaller
- Whole terms can only be written in one block: if it does not fit, it goes into the next block
- Workers that fill the blocks try to detect if there is a reader waiting for them
- Obtain a lock of the block before filling the block; if not, it means the reading thread already has the lock which means that it is processing the block which means it wants small terms from the current filling block, so we move on to the next block in writing thread
- There is always a minimum number of terms in the block - `MINIMUMNUMBEROFTERMS`

Some results after the changes

- No decrease in performance for any tests;
- Performance improvement for benchmarks where term merging is expensive (PolyRatFun, PolyRatFunExpand)
- Forcer, mbox1l